UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação - DC
Graduação em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática 1.1 - Manipulando *Threads*

1. Criar threads com lógicas diferentes.

- Classe ThreadSoma.java

```java
public class ThreadSoma implements Runnable {

	Double n1, n2;

	public ThreadSoma(Double p1, Double p2) {
		this.n1 = p1;
		this.n2 = p2;
	}

	@Override
	public void run() {
		System.out.println(Thread.currentThread().getName()
				+ ": " + "soma: " + (this.n1 + this.n2));

	}

}
```

- Classe ThreadSub.java

```java
public class ThreadSub implements Runnable {

	Double n1, n2;

	public ThreadSub(Double p1, Double p2) {
		this.n1 = p1;
		this.n2 = p2;
	}

	@Override
	public void run() {
		System.out.println(Thread.currentThread().getName()
				+ ": " + "subtração: " + (this.n1 - this.n2));
	}

}
```

- Classe ThreadProduto.java

Java

```
public class ThreadProduto implements Runnable {

	Double n1, n2;

	public ThreadProduto(Double p1, Double p2) {
		this.n1 = p1;
		this.n2 = p2;
	}

	@Override
	public void run() {
		System.out.println(Thread.currentThread().getName()
				+ ": " + "produto: " + (this.n1 * this.n2));

	}

}
```

- Classe ThreadDiv.java

Java

```
public class ThreadDiv implements Runnable {

	Double n1, n2;

	public ThreadDiv(Double p1, Double p2) {
		this.n1 = p1;
		this.n2 = p2;
	}

	@Override
	public void run() {

		System.out.println(Thread.currentThread().getName()
				+ ": " + "divisão: " + (this.n1 / this.n2));

	}

}
```

- Classe App.java

Java

```
public class App {

	public static void main(String[] args) {

		Thread t0 = new Thread(new ThreadSoma(10.0, 20.0));
		t0.start();

		/**
		 * Cria mais três threads em t1, t2 e t3.
```

```
             * Perceba que cada thread é criada a partir
             * da thread principal.
             * Além disso, cada uma tem uma funcionalidade específica.
             * Threads podem ter:
             * Códigos iguais e operar nos mesmos dados.
             * Códigos iguais e operar em dados diferentes.
             * Códigos diferentes e operar sobre os mesmos dados.
             * Códigos diferentes e operar sobre dados diferentes.
             */

            Thread t1 = new Thread(new ThreadSub(10.0, 20.0));
            t1.start();

            Thread t2 = new Thread(new ThreadProduto(10.0, 20.0));
            t2.start();

            Thread t3 = new Thread(new ThreadDiv(10.0, 20.0));
            t3.start();

        }

}
```

2. Brincando com o escalonador e o método *sleep*.

- Classe ThreadCont.java

```
Java
public class ThreadCont implements Runnable{

        private Integer min, max;

        public ThreadCont(Integer min, Integer max) {
                this.min = min;
                this.max = max;
        }

        @Override
        public void run() {

                String nome = Thread.currentThread().getName();

                for (int i = this.min; i < this.max; i++) {
                        System.out.println(nome + " escreve: " + i);
                }

        }

}
```

- Classe App.java

Java

```java
public class TesteThreadCont {

	public static void main(String[] args) {

		Runnable r0 = new ThreadCont(0, 1000);
		Thread t0 = new Thread(r0);
		Runnable r1 = new ThreadCont(0, 1000);
		Thread t1 = new Thread(r1);
		Runnable r2 = new ThreadCont(0, 1000);
		Thread t2 = new Thread(r2);
		Runnable r3 = new ThreadCont(0, 1000);
		Thread t3 = new Thread(r3);
		Runnable r4 = new ThreadCont(0, 1000);
		Thread t4 = new Thread(r4);

		t0.start();
		t1.start();
		t2.start();
		t3.start();
		t4.start();

	}

}
```

- Observe a execução das threads. Qual a ordem de escrita na tela?
- A seguir, a classe ThreadCont.java com o método *sleep*.

Java

```java
public class ThreadCont implements Runnable{

	private Integer min, max, t;

	public ThreadCont(Integer min, Integer max, Integer t) {
		this.min = min;
		this.max = max;
		this.t = t;
	}

	@Override
	public void run() {

		String nome = Thread.currentThread().getName();

		for (int i = this.min; i < this.max; i++) {
			System.out.println(nome + " escreve: " + i);
			try {
				Thread.sleep(t);
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}

		System.out.println(nome + " acabou...");
```

```
	}

}
```

3. Método join().

- Quando uma thread chama join(), a execução do programa principal ou de outra thread que chama join() será bloqueada até que a thread que o chamou termine sua execução.
- Eis um exemplo usando a classe `MinhaThread.java`.

Java

```java
public class MinhaThread implements Runnable{

	@Override
	public void run() {

		for (int i = 1; i <= 5; i++) {
			System.out.println("Thread " + Thread.currentThread().getName()
+ " em execução: " + i);
			try {
				Thread.sleep(500);
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}

	}

}
```

Java

```java
public class ExemploJoin {
	public static void main(String[] args) {

	MinhaThread mt = new MinhaThread();
	Thread t0 = new Thread(mt);
	t0.start();

	try {
	// A thread principal aguardará a conclusão da minhaThread
	t0.join();
	} catch (InterruptedException e) {
		e.printStackTrace();
	}

	System.out.println("Thread principal concluída.");
  }
}
```

- Exemplo com mais threads chamando join().

Java

```java
public class ExemploJoin {
    public static void main(String[] args) {
        MinhaThread mt = new MinhaThread();
        Thread t0 = new Thread(mt);
        t0.start();
        Thread t1 = new Thread(mt);
        t1.start();
        Thread t2 = new Thread(mt);
        t2.start();
        Thread t3 = new Thread(mt);
        t3.start();

        try {
            // A thread principal aguardará a conclusão da minhaThread
            t0.join();
            t1.join();
            t2.join();
            t3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Thread principal concluída.");
    }
}
```

- Exemplo com MinhaThread recebendo um tempo no construtor.

Java

```java
public class MinhaThread implements Runnable{

	Integer tempo;

	public MinhaThread(Integer t) {
		tempo = t;
	}

	@Override
	public void run() {

		for (int i = 1; i <= 5; i++) {
			System.out.println("Thread " +
				Thread.currentThread().getName() +
				" em execução: " +
				i);
			try {
				Thread.sleep(tempo);
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}
```

```
        }
}
```

- Classe ExemploJoin modificada.

```java
public class ExemploJoin {

  public static void main(String[] args) {

        Thread t0 = new Thread(new MinhaThread(200));
        t0.start();
        Thread t1 = new Thread(new MinhaThread(500));
        t1.start();
        Thread t2 = new Thread(new MinhaThread(800));
        t2.start();
        Thread t3 = new Thread(new MinhaThread(1100));
        t3.start();

        try {
               t0.join();
               t1.join();
               t2.join();
               t3.join();
        } catch (InterruptedException e) {
               e.printStackTrace();
        }

        System.out.println("Thread principal concluída.");
  }
}
```

4. Exercícios.

4.1. Criando e manipulando threads: Crie um programa Java que simule uma fábrica de robôs. Cada robô precisa passar por três estágios de produção: Montagem, Teste e Pintura. Para isso, crie três threads que executam essas tarefas em paralelo. Cada thread deve imprimir quando inicia e termina sua execução. Use Thread.sleep() para simular o tempo gasto em cada etapa.

```java
public class Robo implements Runnable {
    private String etapa;

    public Robo(String etapa) {
        this.etapa = etapa;
    }

    @Override
    public void run() {
```

```
        System.out.println("Iniciando " + etapa);
        try {
            Thread.sleep(2000); // Simula o tempo gasto no estágio
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Finalizando " + etapa);
    }

    public static void main(String[] args) {
        Thread montagem = new Thread(new Robo("Montagem"));
        Thread teste = new Thread(new Robo("Teste"));
        Thread pintura = new Thread(new Robo("Pintura"));

        montagem.start();
        teste.start();
        pintura.start();
    }
}
```

4.2. Usando join para sincronização: Crie um programa Java que simule o funcionamento de um elevador em um prédio de três andares. O elevador deve subir do térreo até o terceiro andar, parando em cada andar por um tempo. Cada andar é representado por uma thread, e a thread do andar seguinte só pode iniciar quando a anterior terminar, garantindo que a sequência de andares seja respeitada.

Java

```
class Andar implements Runnable {
    private int numero;

    public Andar(int numero) {
        this.numero = numero;
    }

    @Override
    public void run() {
        System.out.println("Elevador parando no andar " + numero);
        try {
            Thread.sleep(1000); // Simula o tempo parado no andar
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Elevador saindo do andar " + numero);
    }

    public static void main(String[] args) {
        Thread andar1 = new Thread(new Andar(1));
        Thread andar2 = new Thread(new Andar(2));
        Thread andar3 = new Thread(new Andar(3));

        try {
            andar1.start();
            andar1.join();
```

```
            andar2.start();
            andar2.join();

            andar3.start();
            andar3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Elevador chegou ao último andar!");
    }
}
```

4.3. Usando join para cálculo paralelo: Crie um programa Java que calcule a soma de um grande vetor de números inteiros, dividindo o trabalho entre duas threads. Cada thread deve calcular a soma de metade do vetor, e a thread principal deve esperar ambas terminarem para calcular o resultado final.

Java

```java
public class SomaParcial implements Runnable {
    private int[] vetor;
    private int inicio, fim;
    private long resultado;

    public SomaParcial(int[] vetor, int inicio, int fim) {
        this.vetor = vetor;
        this.inicio = inicio;
        this.fim = fim;
    }

    @Override
    public void run() {
        resultado = 0;
        for (int i = inicio; i < fim; i++) {
            resultado += vetor[i];
        }
    }

    public long getResultado() {
        return resultado;
    }

    public static void main(String[] args) {
        int tamanho = 1000000;
        int[] vetor = new int[tamanho];

        // Preenchendo o vetor com valores de 1 a 1000000
        for (int i = 0; i < tamanho; i++) {
            vetor[i] = i + 1;
        }

        SomaParcial soma1 = new SomaParcial(vetor, 0, tamanho / 2);
        SomaParcial soma2 = new SomaParcial(vetor, tamanho / 2, tamanho);
```

```
        Thread t1 = new Thread(soma1);
        Thread t2 = new Thread(soma2);

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        long resultadoFinal = soma1.getResultado() + soma2.getResultado();
        System.out.println("Soma total do vetor: " + resultadoFinal);
    }
}
```